DESIGN DOC · PLANNING PHASE

Prime Tracker Blueprint

Architecture plan for a Warframe prime-part inventory tracker, built as a paired learning project for Docker and Terraform.

● Status: Planning — no code written yet Stack: FastAPI · React · Postgres

Infra: Oracle Cloud Free Tier VM via Terraform Budget: $0 — no paid services

---

CONTENTS

01 The goal

02 Data model

03 Backend API surface

04 Frontend flows

05 Containers & images

06 Terraform & infrastructure

07 Sourcing item data

08 Open decisions

09 Future additions

01 The goal

The app answers one question at a glance, per prime Warframe or weapon: *do I own enough parts to sell this as a complete set, or only pieces I'd sell individually?* Everything below is scoped around that single piece of logic — a user's owned part quantities compared against what a given prime item actually requires.

The secondary goal is the pairing this project was chosen for: Docker builds and runs the application as containers from images, and Terraform provisions the single cloud server those containers run on. The app is intentionally small so the infrastructure work stays the point, not an afterthought bolted onto a big app.

02 Data model

Four entities carry the whole app. A prime item defines what it needs; a user's inventory defines what they have; the comparison between the two is computed, never stored.

ENTITY	KEY FIELDS	PURPOSE
`User`	id, username, email, password_hash	Owns one inventory. Multi-user, so every query is scoped to the authenticated user.
`PrimeItem`	id, name, item_type `(warframe | weapon)`	A tradable prime, e.g. "Volt Prime" or "Braton Prime". One row per item.
`PartRequirement`	id, prime_item_id, part_name, quantity_required	One row per part a given item needs (Neuroptics, Chassis, Barrel, Blueprint…), and how many. Some weapons need more than one of a part — quantity is per-row, not assumed to be 1.
`InventoryEntry`	id, user_id, part_requirement_id, quantity_owned	How many of a specific part a specific user owns. Links straight to the requirement row it's measured against.

Deriving set status

For each `PrimeItem`, join its `PartRequirement` rows against the user's `InventoryEntry` rows (treating a missing entry as zero owned):

- **Complete — every part meets its required quantity**
- **Partial — at least one part owned, but not enough for every requirement**
- **None — zero parts owned toward this item**

This status is computed at query time in the API, not stored — it always reflects current inventory with no risk of going stale.

03 Backend API surface

FastAPI, with JWT-based auth. Every route below except registration and login requires a valid token and operates only on that user's own inventory.

METHOD & PATH	DOES
`POST /auth/register`	Create an account.

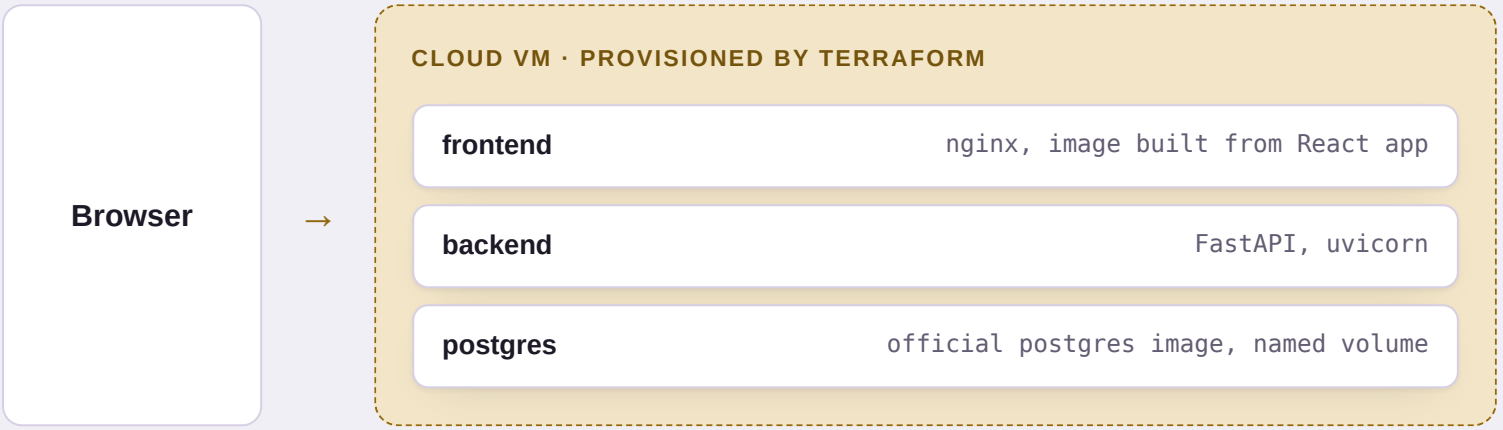
METHOD & PATH	DOES
`POST /auth/login`	Exchange credentials for a JWT.
`GET /items`	List all prime items with their part requirements — the static catalog.
`GET /inventory`	The current user's owned quantities, by part.
`PUT /inventory/{part_requirement_id}`	Set how many of that part the user owns.
`GET /sets`	Every prime item annotated with the current user's status: complete / partial / none.

04 Frontend flows

A small React app with three screens. **Login / register** is the entry point for a new session. The **inventory screen** lets a user search or filter parts and update owned quantities — this is the screen they'll actually use most, so it should be fast to update in bulk rather than one field at a time. The **sets dashboard** is the payoff: prime items grouped by status, complete sets surfaced first since those are what's immediately sellable, with partial sets showing exactly which parts are still missing.

05 Containers & images

Three services, composed together for local development and reused as-is on the deployed VM.



Directory layout:

```
warframe-tracker/
├── backend/
│   ├── app/  # models, routes, auth, seed script
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   └── Dockerfile  # multi-stage: build, then serve via nginx
├── terraform/
│   ├── main.tf, variables.tf, outputs.tf
│   └── user_data.sh  # installs Docker on the VM, runs compose
├── docker-compose.yml
└── .env.example
```

06 Terraform & infrastructure

One VM, kept deliberately simple: Terraform's job is provisioning the box and handing it a startup script, not orchestrating containers directly. That keeps the two tools' responsibilities clean — Docker owns "how the app runs," Terraform owns "what it runs on."

Provider is **Oracle Cloud Infrastructure**, specifically its Always Free Ampere A1 allocation — a real VM (2 OCPU / 12 GB RAM) that stays free indefinitely, not on a countdown. This was chosen over the alternatives below specifically because this is a practice project with a hard $0 budget, not a "free for now" one.

RESOURCE	ROLE
Compute instance	`oci_core_instance` — an Ampere A1 (ARM) VM on the Always Free allocation.
VCN + subnet	`oci_core_vcn` / `oci_core_subnet` — the private network the instance lives in.
Security list	Opens 22 (SSH) and 80/443 (app traffic) only — OCI's equivalent of a security group.
Cloud-init / startup script	Installs Docker + Compose, pulls the repo or images, runs `docker compose up -d`.
Output	The VM's public IP, printed after `terraform apply`.

WORTH KNOWING GOING IN

Oracle's free ARM instances occasionally hit "out of capacity" errors on first provisioning in busy regions — usually resolved by retrying or trying a different availability domain. Also confirm ARM-compatible base images for the Docker builds (multi-arch or ARM-native), since this is an ARM VM, not x86.

07 Sourcing item data

The prime item and part catalog gets seeded from the community-maintained WFCD `warframe-items` dataset rather than typed in by hand — it already has structured data for every prime and its parts, which saves modeling hundreds of items manually.

VERIFY BEFORE SEEDING

Part-quantity requirements aren't uniform — most warframe parts need exactly one of each, but some weapons require more than one of a given part. Before the seed script is written, spot-check quantities for a handful of items against the official Warframe wiki (`wiki.warframe.com`) rather than assuming the dataset's shape without a check.

In practice, seeding starts smaller than the full dataset — a five-item curated set chosen to cover both the normal case and the duplicate-part-quantity edge case. See the resolved decision in Open decisions for the exact list.

08 Open decisions

✓ **Which cloud provider hosts the VM? — Resolved: Oracle Cloud Free Tier**

This is a practice project with a firm $0 budget, so the deciding factor was "genuinely free forever," not "free right now." That ruled out the alternatives:

✕ **Cloudflare** (the user's only prior cloud experience) — Cloudflare Containers requires the paid Workers plan ($5/mo minimum); no free tier exists for running a container there.

✕ **AWS** — as of the July 2025 Free Tier overhaul, new accounts get $100–200 in credit that lasts at most 6 months, then the account either auto-closes or starts billing. Never a permanent free lane, even under the older 12-month model.

✕ **Google Cloud** — its Always Free e2-micro is genuinely perpetual, and stays on the table as a fallback, but its 1 GB of RAM is tight for running Postgres, the API, and the frontend together on one box.

Oracle's Always Free Ampere A1 allocation (2 OCPU / 12 GB RAM) is the only option that's both perpetually free and has enough headroom for all three containers. Cloudflare stays in the picture in a role it's actually free for: DNS in front of the VM, if a custom domain gets added later.

✓ Full WFCD dataset, or a curated subset? — Resolved: curated subset first

Start with a small, deliberately varied set of items — enough to exercise every shape the part-quantity logic needs to handle — and confirm the completion logic is correct before importing the full catalog. Picked specifically to cover the edge case that matters most: most parts need only 1, but some weapons need 2 of the same part, per the official wiki's Duplicate Prime Parts Required page.

ITEM	TYPE	PARTS REQUIRED	WHY IT'S IN THE SEED SET
Volt Prime	Warframe	Neuroptics ×1, Chassis ×1, Systems ×1, Blueprint ×1	Baseline case — every quantity is 1.
Mag Prime	Warframe	Neuroptics ×1, Chassis ×1, Systems ×1, Blueprint ×1	A second frame with the same shape, to confirm items aren't accidentally hardcoded to one row.
Braton Prime	Weapon	Barrel ×1, Receiver ×1, Stock ×1, Blueprint ×1	A weapon where every quantity is still 1 — the "normal" weapon case.
Fang Prime	Weapon	Blade ×2, Handle ×2, Blueprint ×1	Duplicate quantities across *two* different part types — the main edge case to validate.
Glaive Prime	Weapon	Blade ×2, Blueprint ×1	Duplicate quantity on a single part type, with no other component — a different shape again.

Five items, but structurally diverse enough that if the "complete / partial / none" logic works for all five, it should generalize cleanly to the full dataset.

✓ Secrets for the VM (DB password, JWT signing key)? — Resolved: generate + OCI Vault

Oracle's Always Free tier includes OCI Vault (150 free secrets, 20 free key versions), so the proper approach costs nothing extra here — no need to trade security for budget.

The flow

Terraform generates the DB password and JWT signing key with `random_password` — never typed by hand, never reused. Each is written straight into an `oci_vault_secret` resource. The VM's startup script fetches them from Vault via the OCI CLI at boot and writes them into the container environment — the actual values never sit in `user_data`, cloud-init logs, or a checked-in file.

Supporting rules

Every Terraform variable and output touching a secret is marked `sensitive = true`, so real values never print in `plan`/`apply` output. Secrets are injected into containers at runtime via `env_file`, never as a Dockerfile `ENV` line — those get baked permanently into the image layer. `terraform.tfstate` and `.tfvars` stay in `.gitignore` alongside `.env`, since applied resources leave real secret values in state whether or not Vault is used upstream — only `.env.example`, with placeholders, gets committed.

Simpler fallback if Vault feels like too much plumbing for v1: generate secrets the same way with Terraform, but write them straight into a `chmod 600 .env` file on the VM instead of routing through Vault. Less proper, but functional, and Vault can be added once the app itself is working.

09 Future additions

Ideas deliberately kept out of v1 — not open questions blocking implementation, just work worth capturing so it isn't lost. Nothing here needs a decision yet.

→ Trade value per part / set NOT STARTED

Show what a part or complete set is actually worth, alongside its completion status. This splits into two problems with very different difficulty, because ducats and platinum behave nothing alike:

Ducat value — straightforward

Ducat values are fixed by Digital Extremes per part, based on rarity, and don't fluctuate. That makes this just more static data — it can be added as another column on `PartRequirement` and seeded alongside everything else, no external dependency required.

Platinum value — the hard part

Platinum prices are entirely player-set on Warframe Market and move constantly — there's no fixed number to seed. Showing it means either calling Warframe Market's API live on each request (adds an external dependency and latency the rest of this app doesn't have), or caching prices with a periodic refresh job and clearly labeling them as "as of [time]" rather than implying real-time accuracy. Worth treating as its own small project once the core tracker is solid, not a v1 feature.